

User & Address Management

A full-stack web application where administrators view and modify user profiles and their associated addresses (one user to many addresses).

- **Backend:** Java 17 / Spring Boot 3 REST API with JWT authentication and an in-memory `ConcurrentHashMap` store.
- **Frontend:** React 19 + Material UI (MUI) single-page app built with Vite and TypeScript.

Requirements

- JDK 17+ (developed and tested on JDK 26)
- Maven 3.9+
- Node.js 20+ and npm

Getting started

Run the two apps in separate terminals.

1. Backend (port 8080)

```
mvn spring-boot:run
```

Or package and run the jar:

```
mvn package  
java -jar target/user-address-api-1.0.0.jar
```

A seed admin is created on startup:

- Email: `admin@example.com`
- Password: `admin123`

2. Frontend (port 5173)

```
cd frontend  
npm install  
npm run dev
```

Open `http://localhost:5173` and sign in with the admin credentials. The Vite dev server proxies `/api/*` to `http://localhost:8080`, so no CORS configuration is needed.

Features

- JWT sign-in with ADMIN/USER roles
- User list page: all users with email, first name, last name, and role
- Add user (via the public register endpoint) and delete user (cascades addresses)
- User detail page: edit the profile (first name, last name, email, role)
- Address management on the same detail page: add, edit, and delete addresses per user
- Guarded routes: unauthenticated visitors are redirected to the login page

The User -> Address flow (design note)

- The **User List** is the entry point: a compact MUI table with a **Manage** action per row.
- **Manage** navigates to a **User Detail** page that keeps everything about one user in one focused place: a profile card at the top, the user's addresses as cards below.
- The 1-to-many relationship is presented directly on this page: each address is its own card with Edit/Delete actions, and an **Add Address** button creates a new one tied to that user.
- All create/edit flows use MUI dialogs, so the user never loses page context.

- State management uses React hooks only (`useState`, `useEffect`, plus a small auth `Context`). Data is refetched after every mutation, which keeps the UI trivially consistent with the server. Expired sessions are handled centrally: any 401 from the API clears the session and returns the user to the login page.
- Navigation is two-way: the detail page has a back action, and the app bar keeps global context (signed-in user, sign out).

API

All endpoints return JSON wrapped in an `ApiResponse` envelope (`{ success, message, data }`).

Method	Path	Auth	Description
POST	<code>/api/auth/register</code>	Public	Create a user (returns JWT)
POST	<code>/api/auth/login</code>	Public	Login, returns a JWT
GET	<code>/api/users</code>	Bearer	List users
GET	<code>/api/users/{id}</code>	Bearer	Get a user
PUT	<code>/api/users/{id}</code>	Bearer, ADMIN	Update a user (partial)
DELETE	<code>/api/users/{id}</code>	Bearer, ADMIN	Delete a user (204, cascades)
GET	<code>/api/addresses/user/{id}</code>	Bearer	List a user's addresses
POST	<code>/api/addresses</code>	Bearer, ADMIN	Create an address for a user
PUT	<code>/api/addresses/{id}</code>	Bearer, ADMIN	Update an address (partial)
DELETE	<code>/api/addresses/{id}</code>	Bearer, ADMIN	Delete an address (204)

Authenticate by sending `Authorization: Bearer <token>`.

Authorization model

Role-based access control is enforced server-side with method security:

- **ADMIN** can manage all users and addresses (create, update, delete).
- **USER** has read-only access to all users and addresses (including their own profile).
- Registration is public and always creates a **USER**; an admin can change roles via the profile edit endpoint.

Roles are carried in the JWT, so role changes take effect at the next login and a demoted admin retains access until their token expires (24 hours). Missing or unknown role claims fail closed: the request is treated as read-only.

Configuration

Environment variables (optional, dev defaults provided):

Variable	Default	Description
<code>APP_JWT_SECRET</code>	placeholder HS256 key	JWT signing secret; override in real deployments.
<code>ADMIN_PASSWORD</code>	<code>admin123</code>	Password for the seeded <code>admin@example.com</code> account.

```
APP_JWT_SECRET="$(openssl rand -base64 48)" ADMIN_PASSWORD="

```

Tests

Backend (JUnit 5, MockMvc, JaCoCo)

```
mvn test      # run the suite
mvn verify    # run the suite + enforce JaCoCo coverage gates
```

Coverage report: `target/site/jacoco/index.html`.

Frontend (Vitest + React Testing Library)

```
cd frontend
npm test           # run tests once
npm run test:watch # watch mode
npm run build      # type-check + production bundle
```

Project structure

```
src/main/java/com/example/useraddressapi/
  config/      Spring Security configuration
  controller/  REST controllers (auth, users, addresses)
  db/          In-memory ConcurrentHashMap repositories
  dto/         Request/response records
  exception/   Domain exceptions + global handler
  security/    JWT filter and entry point
  service/     Business logic (auth, user, address, jwt)

frontend/src/
  api/         Typed API client and shared types
  components/  RequireAuth, dialogs, AddressCard
  context/     AuthContext (session state)
  pages/       LoginPage, UserListPage, UserDetailPage, AboutPage
```